Experiment No: 4

Student Name: Prionti Das	**UID:** 24BAI71005
Branch: AIT-CSE	**Section/Group:** 24AIT_NTPP-3[B]
Semester: 5TH	**Date of Performance:**
Subject Name: Competitive Coding – II	**Subject Code:** 24CSP-305

Experiment 4.1

Aim: To implement a C++ program to check whether a singly linked list is a **palindrome** using the two-pointer technique and linked-list reversal.

Objective:

- To find the middle of a linked list using **slow and fast pointers**.
- To reverse the second half of the linked list.
- To compare both halves to determine whether the list is a palindrome.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/palindrome-linked-list/submissions/2143469797/

Given the head of a singly linked list, determine whether the linked list reads the same forward and backward. Return true if it is a palindrome; otherwise, return false.

Algorithm:

- Initialize two pointers, slow and fast, at the head.
- Move slow one step and fast two steps at a time to find the middle of the list.
- Reverse the second half of the linked list.
- Set one pointer at the head and another at the beginning of the reversed second half.
- Compare the values of both pointers:
  1. If any values differ, return false.
  2. Otherwise, continue until the second half ends.
- Return true if all corresponding values match.

Code:

```
class Solution {
public:
   bool isPalindrome(ListNode* head) {
      if (!head || !head->next) return true;

      // Step 1: Find the middle of the linked list
      ListNode* slow = head;
      ListNode* fast = head;
      while (fast && fast->next) {
         slow = slow->next;
         fast = fast->next->next;
      }

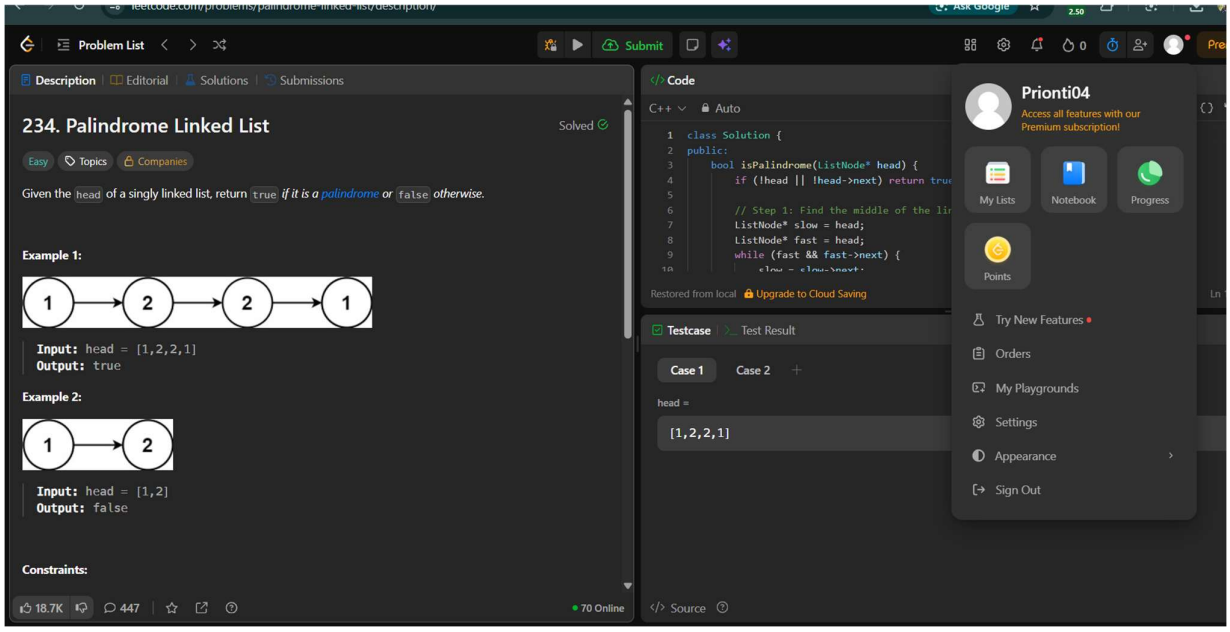
      // Step 2: Reverse the second half of the list
      ListNode* prev = nullptr;
```

```
        ListNode* curr = slow;
        while (curr…   curr = nextNode;
        }

        // Step 3: Compare the first half and the reversed second half
        ListNode* firstHalf = head;
        ListNode* secondHalf = prev; // 'prev' is now the head of the reversed second half
        while (secondHalf) {
            if (firstHalf->val != secondHalf->val) {
                return false;
            }
            firstHalf = firstHalf->next;
            secondHalf = secondHalf->next;
        }

        return true;
    }
};
```

Observations/Outcome: The program successfully determines whether a singly linked list is a palindrome using **O(1) extra space**.



Time Complexity: O(n).

Space Complexity: O(1).

Learning outcomes:

- Learned to use the **slow and fast pointer technique**.
- Learned how to reverse a linked list in-place.
- Understood how to compare two halves of a linked list.
- Learned how to solve a palindrome problem with **linear time and constant extra space**.

Experiment 4.2

Aim: To implement a C++ program to rearrange a linked list by grouping nodes at **odd positions first**, followed by nodes at **even positions**.

Objective:

- To understand linked list pointer manipulation.
- To separate odd and even positioned nodes.
- To rearrange the linked list without using extra storage.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/odd-even-linked-list/submissions/2143483423/

Given the head of a singly linked list, rearrange the nodes so that all nodes at odd indices appear first, followed by all nodes at even indices, while maintaining their original relative order.

Algorithm:

- If the list is empty or contains only one node, return head.
- Set odd to the first node and even to the second node.
- Store the starting even node in evenHead.
- Traverse the linked list while even nodes are available.
- Connect the odd nodes together and move the odd pointer forward.
- Connect the even nodes together and move the even pointer forward.
- Connect the last odd node to evenHead.
- Return the rearranged linked list.

Code:

```
class Solution {
public:
   ListNode* oddEvenList(ListNode* head) {

      if (!head || !head->next) {
         return head;
      }

      ListNode* odd = head;
      ListNode* even = head->next;
      ListNode* evenHead = even;

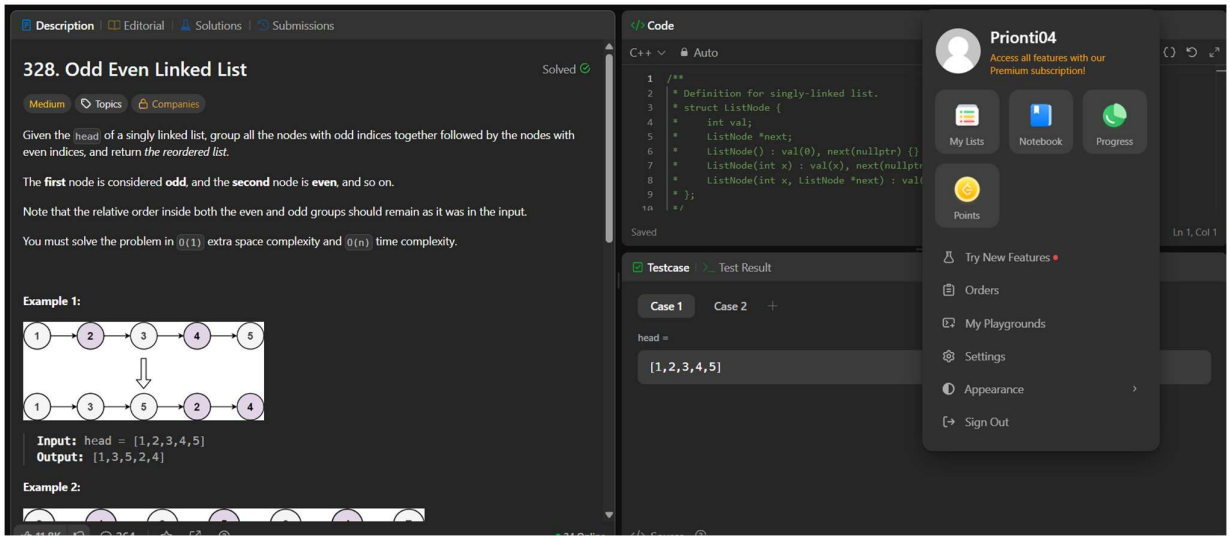
      while (even && even->next) {
         odd->next = even->next;
         odd = odd->next;

         even->next = odd->next;
         even = even->next;
      }

      odd->next = evenHead;

      return head;
   }
};
```

Observations/Outcome: The program successfully rearranges the linked list by placing **odd-positioned nodes before even-positioned nodes** while preserving their relative order.



Time Complexity: O(n)

Space Complexity: O(1)

Learning outcomes:

- Learned how to manipulate pointers in a singly linked list.
- Understood how to separate **odd and even indexed nodes**.
- Learned to rearrange nodes without using extra memory.
- Improved understanding of efficient linked list traversal.